

Facultad de Ingeniería

Ingeniería Civil Informática

**PLATAFORMA DE PREDICCIÓN DE
DEMANDA Y
REABASTECIMIENTO AUTOMÁTICO
PARA
EMPRENDIMIENTOS Y NEGOCIOS EN
CRECIMIENTO
MEDIANTE SELECCIÓN
AUTOMÁTICA DE MODELOS (AMS)**

Tesis para optar al título de Ingeniero Civil en Informática

Autores:

Matias Muñoz Parraguirre

Ignacio Duarte Miranda

Cristóbal Flores Villegas

Profesor guía:

Paulo Luis Francisco Quinsacara Jofré

Santiago, Chile

27 de mayo de 2026

Índice de Contenidos

Resumen	7
Abstract	8
1. Introducción	9
1.1. Contexto	9
1.2. Planteamiento del Problema	10
1.3. Hipótesis	10
1.4. Importancia del Proyecto	10
1.5. Contribución del Proyecto	11
1.6. Objetivos	11
1.6.1. Objetivo General	11
1.6.2. Objetivos Específicos	12
1.7. Limitaciones del Proyecto	12
2. Marco Teórico	14
2.1. Metodologías de Trabajo	14
2.1.1. Metodología Scrum	14
2.1.2. ¿Cómo funciona Scrum?	14
2.1.3. Roles de Scrum	15
2.1.4. Fases de una Metodología Scrum	15
2.1.5. Arquitectura de Software y Modelo de Vistas 4+1	15
2.1.6. Lenguaje de Modelado Gráfico (UML)	16
2.1.7. BPMN	16
2.2. Proceso CRISP-DM	16
2.3. Modelos de Forecasting	17
2.3.1. ARIMA	17
2.3.2. Prophet	17
2.3.3. XGBoost	17
2.3.4. LSTM	17
2.4. Selección Automática de Modelos (AMS)	18

2.5. Gestión de Inventario	18
2.5.1. Modelo EOQ	18
2.5.2. Política (s, S)	19
2.6. Herramientas de Desarrollo	19
2.6.1. Python 3.11	19
2.6.2. FastAPI	19
2.6.3. React y Vite	19
2.6.4. PostgreSQL y Supabase	20
2.6.5. Claude API (Anthropic)	20
2.6.6. PyTorch	20
2.6.7. Pandas y Bibliotecas Científicas	20
2.7. Multi-tenancy y Arquitectura SaaS	20
2.8. Dataset: Store Sales — Corporación Favorita	21
3. Identificación de Requisitos	22
3.1. Partes Interesadas (Stakeholders)	22
3.2. Usuarios	22
3.3. Roles	22
3.4. Requisitos del Sistema	23
3.5. Planificación Sprint	26
4. Product Backlog y Sprint Backlog	27
4.1. Product Backlog	27
4.2. Casos de Uso	28
4.3. Sprint Backlog — Sprint 1	31
4.4. Sprint Backlog — Sprint 2	31
4.5. Sprint Backlog — Sprint 3	32
4.6. Sprint Backlog — Sprint 4	33
4.7. Sprint Backlog — Sprint 5	34
5. Modelos de Vistas 4+1	35
5.1. Vista Lógica — Diagrama de Clases	35
5.2. Vista Lógica — Diagrama de Base de Datos	35
5.3. Vista de Procesos — Diagrama de Secuencia	36
5.4. Vista de Desarrollo — Diagrama de Componentes	37
5.5. Vista Física — Diagrama de Despliegue	37
6. Desarrollo	38
6.1. Pipeline ETL	38
6.2. Módulo de Forecasting	38

6.3. Módulo de Inventario	38
6.4. Módulo de Ingesta Inteligente	39
6.4.1. Flujo de Ingesta	39
6.4.2. Resolución de Solapamiento entre Cargas	40
6.5. Asistente Conversacional Stocky	40
6.6. API REST	40
6.7. Dashboard Web	41
6.8. Vistas del Sistema	42
7. Pruebas y Resultados	44
7.1. Pruebas Unitarias	44
7.2. Resultados del Módulo AMS	47
7.2.1. Datos Reales — Distribuidora Santa Elena	47
7.2.2. Comparativa AMS vs. Modelo Único	48
8. Retrospectiva del Sprint	49
8.1. Comprobación de Sprint 1 — Fundamentos y ETL	49
8.2. Retrospectiva del Sprint 1	49
8.3. Comprobación de Sprint 2 — Autenticación, Forecasting y Validador	50
8.4. Retrospectiva del Sprint 2	50
8.5. Comprobación de Sprint 3 — Gestión de Cargas y Auditoría	51
8.6. Retrospectiva del Sprint 3	51
8.7. Comprobación de Sprint 4 — Inventario, Órdenes y Stocky	51
8.8. Retrospectiva del Sprint 4	52
8.9. Comprobación de Sprint 5 — LSTM PyTorch y Preparación Deploy	52
8.10. Retrospectiva del Sprint 5	52
8.11. Gráficos Burn-down y Burn-up	53
8.12. Diagrama BPMN — Flujo de Ingesta Inteligente	53
9. Conclusiones	54
9.1. Conclusiones respecto a los Objetivos	54
9.1.1. Objetivos Específicos	54
9.1.2. Objetivo General	55
9.2. Beneficios del Sistema	55
9.3. Limitaciones	55
9.4. Conclusión respecto a la Metodología Scrum	56
9.5. Proyecciones	56
Referencias	57

Índice de Figuras

2.1. Proceso Scrum — ciclo de sprints	14
5.1. Diagrama de Clases de SmartSupply	35
5.2. Diagrama de Base de Datos de SmartSupply	36
5.3. Diagrama de Secuencia — Predicción de demanda via AMS	36
5.4. Diagrama de componentes de SmartSupply	37
5.5. Diagrama de despliegue de SmartSupply (Cloudflare Pages + Render + Supabase)	37
6.1. Vista: Dashboard con KPIs reales del negocio	42
6.2. Vista: Forecasting con predicción del modelo ganador AMS	42
6.3. Vista: Ingesta Inteligente con validador de calidad	42
6.4. Vista: Asistente Stocky respondiendo consulta de inventario	43
8.1. Burn-down por sprint y Burn-up acumulado del proyecto (Sprints 1–5)	53
8.2. BPMN: Proceso de ingesta de datos con validación IA	53

Índice de Tablas

2.1. Roles Scrum — SmartSupply	15
2.2. Características del Dataset Store Sales — Corporación Favorita	21
3.1. Stakeholders de SmartSupply	22
3.2. Requisito RF-01: Autenticación de Usuarios	23
3.3. Requisito RF-02: Registrar Negocio en el Sistema	23
3.4. Requisito RF-03: Ingestar Datos de Ventas mediante IA	24
3.5. Requisito RF-04: Ejecutar Predicción de Demanda por SKU	24
3.6. Requisito RF-05: Generar Alerta de Reabastecimiento	24
3.7. Requisito RF-06: Crear Orden de Compra Automática	25
3.8. Requisito RF-07: Visualizar Dashboard de KPIs	25
3.9. Requisito RF-08: Asistente Conversacional (Stocky)	25
3.10. Planificación de Sprints — SmartSupply	26
4.1. Product Backlog — SmartSupply	27
4.2. Caso de Uso CU-01: Acceder al Sistema	28
4.3. Caso de Uso CU-02: Cargar Datos de Ventas mediante IA	29
4.4. Caso de Uso CU-03: Consultar Predicción de Demanda	29
4.5. Caso de Uso CU-04: Generar Órdenes de Compra Automáticas	30
4.6. Caso de Uso CU-05: Consultar Asistente Stocky	30
4.7. Sprint Backlog — Sprint 1: Setup, ETL y Estructura Base	31
4.8. Sprint Backlog — Sprint 2: Auth JWT + Forecasting Real	31
4.9. Sprint Backlog — Sprint 3: Gestión de Datos y Cargas	32
4.10. Sprint Backlog — Sprint 4: Inventario, Órdenes y Stocky	33
4.11. Sprint Backlog — Sprint 5: Dashboard KPIs + LSTM DIRMO + Deploy	34
6.1. Principales endpoints de la API REST	41
7.1. Prueba Unitaria PU-01: Predicción de Demanda AMS	44
7.2. Prueba Unitaria PU-02: Alertas de Inventario	45
7.3. Prueba Unitaria PU-03: Ingesta IA — Preview	45
7.4. Prueba Unitaria PU-04: Ingesta IA — Confirmación y Auditoría	46

7.5. Prueba Unitaria PU-05: Autenticación JWT	46
7.6. Prueba Unitaria PU-06: Asistente Stocky — Loop Agentivo	47
7.7. Resultados AMS — Familia PANADERÍA Y PASTELERÍA (Distribuidora Santa Elena)	47
7.8. Comparativa: AMS vs. modelo único ARIMA (baseline)	48
8.1. Recuento de Tareas — Sprint 1	49
8.2. Recuento de Tareas — Sprint 2	50
8.3. Recuento de Tareas — Sprint 3	51
8.4. Recuento de Tareas — Sprint 4	51
8.5. Recuento de Tareas — Sprint 5	52

Resumen

La gestión del inventario y la predicción de demanda representan uno de los mayores desafíos operacionales para emprendimientos y negocios en crecimiento. Las soluciones existentes —como SAP o Oracle— son costosas y requieren implementaciones largas, quedando fuera del alcance de la mayoría de las pequeñas y medianas empresas. A su vez, la diversidad de comportamientos de demanda entre productos dificulta la aplicación de un único modelo de predicción para todos los SKUs.

El presente proyecto propone **SmartSupply**, una plataforma SaaS de predicción de demanda y reabastecimiento automático que implementa un mecanismo de Selección Automática de Modelos (AMS) por SKU, evaluando cuatro algoritmos —ARIMA, Prophet, XGBoost y LSTM— y seleccionando aquel con menor error de predicción (WAPE) para cada familia de productos. La hipótesis central plantea que esta selección automática reduce el error de predicción y el capital inmovilizado frente al uso de un modelo único.

La plataforma integra un pipeline ETL sobre el dataset público *Store Sales — Corporación Favorita* (Kaggle, 2017) para benchmarking, una API REST construida con FastAPI, un módulo de gestión de inventario basado en las políticas EOQ y (s, S) , y un dashboard web en React. El sistema incorpora un **módulo de ingesta inteligente** que utiliza el modelo Claude (Anthropic) con capacidades de visión para extraer y normalizar datos de ventas desde imágenes, planillas Excel o PDFs, eliminando la barrera de entrada para negocios sin sistemas digitalizados. La arquitectura es **multi-tenant** y escalable: cada negocio opera de forma aislada sobre la misma infraestructura mediante `business_id`.

Los resultados obtenidos sobre datos reales muestran que el AMS selecciona el modelo óptimo por SKU de forma consistente, con errores WAPE por debajo del 15 % en series con patrones regulares. La metodología de desarrollo sigue el framework ágil Scrum en 5 sprints y el proceso CRISP-DM para el tratamiento de datos.

Palabras clave: predicción de demanda, selección automática de modelos, WAPE, inventario, EOQ, política (s, S) , ARIMA, Prophet, XGBoost, LSTM, FastAPI, React, ingesta IA, multi-tenancy, emprendimientos, SaaS.

Abstract

Inventory management and demand forecasting represent one of the greatest operational challenges for startups and growing businesses. Existing enterprise solutions —such as SAP or Oracle— are cost-prohibitive and require lengthy implementations, placing them out of reach for most small and medium-sized enterprises. Furthermore, the diversity of demand behaviors across product families makes it difficult to apply a single forecasting model to all SKUs.

This project proposes **SmartSupply**, a SaaS platform for demand forecasting and automatic restocking that implements an Automatic Model Selection (AMS) mechanism per SKU, evaluating four algorithms —ARIMA, Prophet, XGBoost and LSTM— and selecting the one with the lowest forecasting error (WAPE) for each product family. The central hypothesis is that this per-SKU automatic selection reduces forecasting error and tied-up capital compared to using a single model for all products.

The platform integrates an ETL pipeline over the public dataset *Store Sales — Corporación Favorita* (Kaggle, 2017) for benchmarking, a REST API built with FastAPI, an inventory management module based on EOQ and (s, S) policies, and a React web dashboard. The system incorporates an **intelligent ingestion module** that uses the Claude language model (Anthropic) with vision capabilities to extract and normalize sales data from images, Excel files or PDFs, eliminating the digital barrier for businesses without ERP systems. The architecture is **multi-tenant** and scalable: each business operates in isolation on the same infrastructure through a unique **business_id**.

Results obtained on real ingested data show that the AMS consistently selects the optimal model per SKU, achieving WAPE values below 15 % on series with regular patterns. The development methodology follows the Scrum agile framework across 5 sprints and the CRISP-DM process for data treatment.

Keywords: demand forecasting, automatic model selection, WAPE, inventory, EOQ, (s, S) policy, ARIMA, Prophet, XGBoost, LSTM, FastAPI, React, AI ingestion, multi-tenancy, startups, SaaS.

1. Introducción

1.1. Contexto

En el ecosistema actual de emprendimientos y pequeñas y medianas empresas (PyMEs), la gestión eficiente del inventario y la predicción de demanda representan desafíos operacionales críticos. Un quiebre de stock implica ventas perdidas y deterioro de la relación con el cliente, mientras que un exceso de inventario inmoviliza capital de trabajo que un emprendimiento en crecimiento no puede permitirse desperdiciar. Ambos escenarios son consecuencia directa de predicciones de demanda imprecisas o metodologías de reabastecimiento estáticas.

Las herramientas empresariales existentes para abordar este problema —como SAP, Oracle NetSuite o Microsoft Dynamics— presentan barreras de entrada significativas en términos de costo, tiempo de implementación y complejidad técnica, quedando fuera del alcance de la mayoría de los emprendimientos y negocios en etapa de crecimiento (Ramírez & Concha, 2021). Esto genera una brecha tecnológica: mientras las grandes corporaciones optimizan su inventario con algoritmos avanzados, las empresas más pequeñas siguen tomando decisiones de compra de forma intuitiva o con hojas de cálculo.

Adicionalmente, muchos emprendimientos no cuentan con datos de ventas en formato digital estructurado. Sus registros históricos existen en cuadernos, planillas Excel desordenadas o comprobantes físicos, lo que impide aplicar cualquier modelo predictivo sin un proceso previo de digitalización y normalización de datos.

La proliferación de algoritmos de *machine learning* y series de tiempo ofrece hoy múltiples alternativas para el pronóstico de ventas, pero la elección del modelo óptimo varía según las características de cada producto (estacionalidad, tendencia, presencia de ceros) y el horizonte de predicción requerido. No existe un único modelo que funcione bien para todos los SKUs de un mismo negocio.

1.2. Planteamiento del Problema

Los emprendimientos y negocios en crecimiento carecen de herramientas accesibles que les permitan: (1) digitalizar su historial de ventas desde cualquier formato, (2) predecir la demanda futura con el modelo estadístico más adecuado para cada producto, y (3) generar automáticamente políticas de reabastecimiento basadas en esas predicciones.

La pregunta de investigación que guía este proyecto es:

¿En qué medida la selección automática del mejor modelo de forecasting por SKU reduce el error de predicción y el capital inmovilizado frente al uso de un modelo único para todos los productos en emprendimientos y negocios en crecimiento?

1.3. Hipótesis

La selección automática del mejor modelo de forecasting por SKU (AMS) mediante validación *walk-forward* reduce el error de predicción global (WAPE) y, en consecuencia, el capital inmovilizado en inventario, en comparación con el uso de un modelo único aplicado a todos los productos de un mismo negocio, independientemente del rubro o tamaño de la empresa.

1.4. Importancia del Proyecto

La selección inadecuada del modelo de forecasting es una de las causas más frecuentes de error en los sistemas de reabastecimiento. Un modelo que funciona bien para productos de alta rotación con estacionalidad semanal puede ser completamente inadecuado para artículos con demanda esporádica o con tendencias de largo plazo. Automatizar esta selección por SKU, de forma transparente y sin requerir conocimiento técnico por parte del usuario, permite:

- Reducir el error de predicción global (WAPE) respecto al uso de un modelo único.
- Disminuir el capital inmovilizado en inventario al calibrar mejor los puntos de reorden.
- Generar órdenes de compra oportunas y calibradas a la demanda real de cada producto.
- Eliminar la barrera de digitalización de datos mediante ingesta IA.

- Entregar a emprendedores y gestores una herramienta visual accesible, sin necesidad de conocimiento en ciencia de datos o sistemas ERP.
- Escalar desde un emprendimiento unipersonal hasta una cadena con múltiples sucursales, gracias a la arquitectura multi-tenant.

1.5. Contribución del Proyecto

SmartSupply realiza los siguientes aportes:

1. Un **módulo AMS** que selecciona automáticamente el mejor modelo de forecasting por SKU mediante validación walk-forward (70 / 15 / 15), evaluando ARIMA, Prophet, XGBoost y LSTM.
2. Un **módulo de inventario** que aplica EOQ y política (s, S) sobre los pronósticos generados por AMS, calculando puntos de reorden y cantidades óptimas de pedido.
3. Una **API REST** (FastAPI) documentada automáticamente via OpenAPI/Swagger, consumible por cualquier sistema externo.
4. Un **dashboard web** (React + Recharts) para visualización de pronósticos, alertas de inventario, órdenes de compra y KPIs operacionales en tiempo real.
5. Un **pipeline ETL** reproducible sobre el dataset Store Sales — Corporación Favorita (Kaggle, 2017) para benchmarking y validación de la hipótesis.
6. Un **módulo de ingesta inteligente** que utiliza Claude (Anthropic) con capacidades de visión y procesamiento de documentos para extraer y normalizar datos de ventas desde imágenes, planillas Excel y PDFs, eliminando la barrera de entrada para negocios sin sistemas digitalizados.
7. Una **arquitectura multi-tenant escalable** que aísla los datos de cada negocio mediante `business_id`, habilitando el despliegue como plataforma SaaS accesible desde un emprendimiento hasta una empresa con múltiples sucursales.

1.6. Objetivos

1.6.1. Objetivo General

Desarrollar una plataforma SaaS de predicción de demanda y reabastecimiento automático para emprendimientos y negocios en crecimiento que, mediante la selección

automática del mejor modelo de forecasting por SKU, reduzca el error de predicción y el capital inmovilizado respecto al uso de un modelo único para todos los productos.

1.6.2. Objetivos Específicos

1. Implementar y evaluar cuatro modelos de predicción de series de tiempo (ARIMA, Prophet, XGBoost y LSTM) sobre el dataset Store Sales — Corporación Favorita, como conjunto de benchmarking representativo de negocios con múltiples SKUs.
2. Desarrollar el mecanismo AMS que selecciona el modelo de menor WAPE por SKU mediante validación cruzada temporal (walk-forward con partición 70/15/15).
3. Implementar las políticas de gestión de inventario EOQ y (s, S) alimentadas por los pronósticos del AMS, generando parámetros de reorden por SKU.
4. Construir una API REST con FastAPI que exponga los módulos de forecasting e inventario, con autenticación JWT y arquitectura multi-tenant por `business_id`.
5. Desarrollar un dashboard web interactivo en React para la visualización de pronósticos, métricas de inventario, gestión de órdenes de compra y KPIs operacionales del negocio.
6. Diseñar e implementar un pipeline ETL que cargue, limpie y normalice el dataset de benchmarking en una base de datos PostgreSQL alojada en Supabase.
7. Desarrollar un módulo de ingesta inteligente basado en IA (Claude, Anthropic) que permita a cualquier negocio incorporar sus datos históricos de ventas desde cualquier formato digital (imágenes, Excel, PDF), con validación de calidad previa a la carga.

1.7. Limitaciones del Proyecto

- La validación cuantitativa de la hipótesis se realiza sobre el dataset Store Sales — Corporación Favorita (Ecuador, 2013–2017), que actúa como negocio de demostración. El módulo de ingesta IA permite incorporar datos reales de cualquier negocio en producción.
- El AMS evalúa cuatro modelos de forecasting; existen alternativas (SARIMA, N-BEATS, Temporal Fusion Transformer) no contempladas en este alcance pero integrables en versiones futuras.
- El módulo de inventario asume demanda independiente por SKU; no modela restricciones de capacidad de bodega ni descuentos por volumen.

- La calidad de la extracción del módulo de ingesta IA depende de la legibilidad del documento fuente; imágenes de muy baja resolución pueden producir resultados incompletos.
- La integración bidireccional con sistemas ERP o WMS externos queda fuera del alcance de esta tesis; la API REST está disponible para integraciones futuras.
- El sistema ha sido validado principalmente con datos de retail y distribución; rubros con demanda muy irregular (servicios bajo pedido, proyectos) pueden requerir ajustes en los modelos.

2. Marco Teórico

2.1. Metodologías de Trabajo

Una metodología de trabajo es un conjunto de técnicas, reglas y procedimientos que estructuran la forma en que un equipo desarrolla un proyecto. Al adoptar una metodología bien definida se mejora la eficiencia, la calidad de los entregables y la comunicación interna del equipo.

2.1.1. Metodología Scrum

Scrum es un framework ágil para el desarrollo iterativo e incremental de productos complejos (Schwaber & Sutherland, 2020). Se organiza en ciclos de trabajo llamados *Sprints* de duración fija —en este proyecto, dos semanas— al final de los cuales se entrega un incremento potencialmente funcional del sistema. Scrum permite que los equipos se autogestionen, aprendan de la experiencia y se adapten a los cambios del entorno, características esenciales en proyectos de software con requerimientos evolutivos.

2.1.2. ¿Cómo funciona Scrum?

El proceso Scrum comienza con la definición del *Product Backlog*, lista priorizada de todas las funcionalidades requeridas. En cada *Sprint Planning* el equipo selecciona los ítems a desarrollar en el siguiente Sprint, generando el *Sprint Backlog*. Diariamente se realiza una reunión breve (*Daily Scrum*) para sincronizar el avance. Al final del Sprint se realiza la *Sprint Review* (demo) y la *Sprint Retrospective* (mejora de proceso).

[Diagrama del proceso Scrum]

Figura 2.1: Proceso Scrum — ciclo de sprints

2.1.3. Roles de Scrum

Para aplicar la metodología Scrum se definen tres roles esenciales: el Product Owner, el Scrum Master y el Equipo de Desarrollo. Cada uno tiene responsabilidades específicas descritas en The Scrum Guide (Schwaber & Sutherland, 2020).

Tabla 2.1: Roles Scrum — SmartSupply

Rol	Responsabilidad	Integrante
Product Owner	Priorizar Product Backlog, validar entregables	Paulo Quinsacara Jofré
Scrum Master	Facilitar ceremonias, remover impedimentos	Matias Muñoz Parraguirre
Dev Team	Diseño, implementación, pruebas	Los 3 integrantes

2.1.4. Fases de una Metodología Scrum

Las fases principales son: Planificación del Sprint, Ejecución del Sprint (con Daily Scrums), Sprint Review y Sprint Retrospective. Este ciclo se repite durante los sprints planificados. En SmartSupply se ejecutaron 5 sprints de desarrollo más un sprint final de QA y despliegue.

2.1.5. Arquitectura de Software y Modelo de Vistas 4+1

El modelo de vistas 4+1 (Kruchten, 1995) describe la arquitectura de un sistema de software a través de cinco perspectivas complementarias:

- **Vista Lógica:** estructura de clases y componentes del dominio.
- **Vista de Procesos:** comportamiento dinámico del sistema en tiempo de ejecución (diagramas de secuencia y actividad).
- **Vista de Desarrollo:** organización del código fuente en módulos, paquetes y capas (diagrama de componentes).
- **Vista Física:** infraestructura de despliegue, servidores y redes (diagrama de despliegue).
- **Vista de Escenarios (“+1”):** casos de uso que integran las cuatro vistas anteriores desde la perspectiva del usuario.

SmartSupply aplica este modelo en el Capítulo 5, documentando cada vista con los diagramas UML correspondientes.

2.1.6. Lenguaje de Modelado Gráfico (UML)

UML (*Unified Modeling Language*) es un estándar de la industria para la visualización, especificación y documentación de sistemas de software orientados a objetos (Booch, Rumbaugh & Jacobson, 1999). En este proyecto se utilizan los diagramas de clases, secuencia, componentes y despliegue para documentar la arquitectura del sistema.

2.1.7. BPMN

BPMN (*Business Process Model and Notation*) es un estándar para el modelado de procesos de negocio mediante una notación gráfica estandarizada (OMG, 2011). Permite representar flujos de trabajo con participantes, eventos, actividades y decisiones, facilitando la comunicación entre equipos técnicos y de negocio. En SmartSupply se utiliza BPMN para modelar el flujo completo del proceso de ingesta inteligente de datos.

2.2. Proceso CRISP-DM

CRISP-DM (*Cross Industry Standard Process for Data Mining*) es un proceso estándar para proyectos de ciencia de datos que consta de seis fases (Shearer, 2000):

1. **Business Understanding:** definir el problema de negocio y los criterios de éxito. En SmartSupply, esto corresponde a reducir el error de predicción y el capital inmovilizado.
2. **Data Understanding:** exploración y análisis de calidad de los datos. Se analizó el dataset Store Sales — Corporación Favorita y los datos reales ingestados por cada negocio.
3. **Data Preparation:** limpieza, transformación e ingeniería de características. Implementado en `02_clean.py` del pipeline ETL.
4. **Modeling:** selección y entrenamiento de modelos. Corresponde al módulo AMS con los cuatro algoritmos de forecasting.
5. **Evaluation:** validación contra métricas de negocio mediante WAPE en ventana walk-forward.
6. **Deployment:** despliegue del sistema en Render (backend) y Cloudflare Pages (frontend).

2.3. Modelos de Forecasting

2.3.1. ARIMA

ARIMA (*Autoregressive Integrated Moving Average*) es un modelo estadístico clásico para series de tiempo estacionarias (Box & Jenkins, 1976). Se define por tres parámetros (p, d, q) : orden autorregresivo, grado de diferenciación y orden de media móvil. En su variante SARIMA se agrega un componente estacional $(P, D, Q)_m$ que captura ciclos de periodo m . En SmartSupply, el AMS realiza una búsqueda en grilla sobre 144 combinaciones de parámetros, incluyendo componente estacional semanal ($m = 7$) cuando la serie cuenta con al menos 21 observaciones.

2.3.2. Prophet

Prophet es un modelo de forecasting desarrollado por Meta (Taylor & Letham, 2018), diseñado para manejar estacionalidades múltiples, efectos de feriados y valores atípicos. Descompone la serie en tendencia, estacionalidad y calendario. En SmartSupply, la estacionalidad anual se activa dinámicamente solo cuando la serie tiene 730 o más días de historia, evitando el sobreajuste en series cortas.

2.3.3. XGBoost

XGBoost (*Extreme Gradient Boosting*) es un algoritmo de ensamble basado en árboles de decisión (Chen & Guestrin, 2016). Para series de tiempo se aplica mediante ingeniería de características temporales: lags de ventas, medias móviles, variables de calendario (día de semana, mes) y features exógenas (feriados). En SmartSupply se implementa con la estrategia DIRMO (*Direct Multi-Output*), donde se entrena un modelo independiente por cada horizonte de predicción, evitando la acumulación de error en predicciones multi-paso.

2.3.4. LSTM

Las redes LSTM (*Long Short-Term Memory*) son una variante de las redes neuronales recurrentes que resuelven el problema del gradiente desvanecido mediante compuertas de entrada, olvido y salida (Hochreiter & Schmidhuber, 1997). Son especialmente efectivas para capturar dependencias de largo plazo en series temporales. En SmartSupply, el modelo LSTM está implementado en PyTorch con la estrategia DIRMO y utiliza seis *features* de entrada: ventas históricas, componentes seno/coseno del día de semana y

del mes, e indicador de feriado chileno. Incluye corrección automática de días de cierre basada en el historial de demanda cero por día de semana.

2.4. Selección Automática de Modelos (AMS)

El AMS (*Automatic Model Selection*) implementa validación *walk-forward*: el historial de ventas de cada SKU se divide en 70 % entrenamiento, 15 % validación y 15 % prueba. Para cada SKU, los cuatro modelos se entrenan y evalúan sobre la ventana de validación; el de menor WAPE se designa como modelo activo para ese SKU y genera las predicciones del horizonte solicitado.

Se utiliza el WAPE (*Weighted Absolute Percentage Error*) en lugar del MAPE clásico, ya que el MAPE es inestable cuando la demanda real y_t se acerca a cero — condición frecuente en series de ventas con días sin actividad. La fórmula del WAPE es:

$$\text{WAPE} = \frac{\sum_{t=1}^n |y_t - \hat{y}_t|}{\sum_{t=1}^n y_t} \times 100 \%$$

donde y_t es el valor real y $\hat{y}_t$ el valor predicho. Al ponderar el error por el volumen total de ventas, el WAPE es más robusto ante valores nulos y refleja mejor el impacto económico del error de predicción.

2.5. Gestión de Inventario

2.5.1. Modelo EOQ

El *Economic Order Quantity* (EOQ) determina la cantidad óptima de pedido que minimiza los costos totales de inventario, combinando el costo de hacer un pedido y el costo de mantener unidades en stock (Harris, 1913):

$$Q^* = \sqrt{\frac{2DK}{h}}$$

donde D es la demanda anual proyectada por el AMS, K el costo fijo por orden de compra y h el costo de mantención por unidad por año.

2.5.2. Política (s, S)

La política de inventario (s, S) establece dos parámetros: el punto de reorden s (nivel mínimo de stock que dispara un pedido) y el nivel objetivo S (nivel al que se repone el inventario tras el pedido). Cuando el stock cae por debajo de s , se genera una orden por $Q = S - \text{stock actual}$ unidades (Silver, Pyke & Peterson, 1998). Los parámetros se calculan como:

$$s = \bar{d} \cdot L + z_\alpha \cdot \sigma_d \cdot \sqrt{L} \quad S = s + Q^*$$

donde $\bar{d}$ es la demanda media diaria del AMS, L el *lead time* en días, σ_d la desviación estándar de la demanda y z_α el factor de servicio.

2.6. Herramientas de Desarrollo

2.6.1. Python 3.11

Python es el lenguaje principal del proyecto, utilizado en el backend, ETL y módulos de forecasting. Se emplea la versión 3.11 por su compatibilidad con las bibliotecas de machine learning utilizadas. Las dependencias se gestionan mediante un entorno virtual (`venv`).

2.6.2. FastAPI

FastAPI es un framework web moderno para Python que genera automáticamente documentación OpenAPI/Swagger a partir de las anotaciones de tipo del código (Ramírez, 2018). Ofrece validación automática de datos mediante Pydantic y rendimiento comparable a frameworks asíncronos de Node.js.

2.6.3. React y Vite

React es una biblioteca JavaScript para construir interfaces de usuario basadas en componentes reutilizables (Meta, 2013). Vite actúa como bundler de desarrollo con recarga en caliente. En SmartSupply el frontend utiliza React 18 con TanStack Query para gestión de estado del servidor, Recharts para visualizaciones y Zustand para estado de autenticación.

2.6.4. PostgreSQL y Supabase

PostgreSQL es el sistema de gestión de base de datos relacional del proyecto. Supabase provee PostgreSQL como servicio gestionado en la nube, incluyendo autenticación, Row Level Security (RLS) y una API REST automática. SmartSupply utiliza la capa de conexión directa vía SQLAlchemy 2.0 sobre el *connection pooler* de Supabase.

2.6.5. Claude API (Anthropic)

Claude es un modelo de lenguaje grande (LLM) desarrollado por Anthropic con capacidades de visión y procesamiento de documentos. En SmartSupply se utilizan dos variantes: `claude-haiku-4-5` para extracción de imágenes y PDFs (menor latencia) y `claude-sonnet-4-6` para mapeo de columnas en planillas Excel con nomenclatura ambigua (mayor razonamiento). La integración se realiza mediante el SDK oficial de Anthropic para Python.

2.6.6. PyTorch

PyTorch es una biblioteca de aprendizaje profundo de código abierto desarrollada por Meta AI. En SmartSupply se utiliza para implementar el modelo LSTM, aprovechando su API dinámica de grafos de cómputo y facilidad de depuración respecto a alternativas basadas en grafos estáticos.

2.6.7. Pandas y Bibliotecas Científicas

Pandas es la biblioteca central para manipulación de datos tabulares en Python. Se complementa con `statsmodels` (ARIMA/SARIMA), `prophet` (Meta Prophet), `xgboost` (algoritmo de ensamble) y `scikit-learn` (métricas y utilidades de validación).

2.7. Multi-tenancy y Arquitectura SaaS

Una arquitectura *multi-tenant* permite que múltiples clientes (tenants) compartan la misma infraestructura de software manteniendo sus datos completamente aislados (Curino et al., 2010). Este modelo es la base de las plataformas SaaS (*Software as a Service*), donde el proveedor opera un único sistema y cada cliente accede a su instancia lógica mediante credenciales y un identificador único.

En SmartSupply, cada negocio registrado recibe un `business_id` único. Todas las tablas de la base de datos (`sales_history`, `products`, `stock_levels`, `purchase_orders`,

`ingest_log`) incluyen `business_id` como clave de partición lógica. Los endpoints de la API filtran automáticamente los datos por el negocio del usuario autenticado, garantizando el aislamiento. Esta arquitectura permite que SmartSupply escale desde un emprendimiento unipersonal hasta una cadena con múltiples sucursales sin cambios en la infraestructura.

2.8. Dataset: Store Sales — Corporación Favorita

El dataset público Store Sales — Corporación Favorita (Kaggle, 2017) contiene datos reales de ventas de 54 tiendas en Ecuador durante el período 2013–2017. Se utiliza en SmartSupply como conjunto de benchmarking para entrenar y evaluar el AMS sobre un volumen de datos representativo de un negocio con múltiples SKUs y sucursales. Sus características se resumen en la Tabla 2.2.

Tabla 2.2: Características del Dataset Store Sales — Corporación Favorita

Característica	Valor
Período	2013-01-01 a 2017-08-15
Filas (train)	3.000.888
Tiendas	54
Familias (SKUs)	33
Columnas	id, date, store_nbr, family, sales, onpromotion
Demanda cero	~31,3 % de los registros
Archivos extra	stores.csv, oil.csv, holidays_events.csv, transactions.csv

3. Identificación de Requisitos

3.1. Partes Interesadas (Stakeholders)

Tabla 3.1: Stakeholders de SmartSupply

Stakeholder	Interés	Influencia
Emprendedor / Dueño del negocio	Controlar inventario, reducir capital inmovilizado	Alta
Encargado de Operaciones	Gestionar órdenes de compra, evitar quiebres de stock	Alta
Analista / Supervisor	Monitorear métricas de predicción y nivel de servicio	Medio
Inversionistas / Dirección	Reducir costos operacionales y mejorar rentabilidad	Alta
Equipo de TI (si existe)	Integración con sistemas existentes vía API REST	Medio

3.2. Usuarios

Los usuarios directos del sistema son:

- **Administrador** (dueño o fundador del negocio): registra el negocio, carga datos históricos de ventas, gestiona productos y parámetros de inventario, y tiene acceso completo a todas las funciones del sistema.
- **Operador** (encargado de operaciones o bodega): consulta alertas de inventario, genera órdenes de compra, monitorea el estado de los pedidos y utiliza el asistente Stocky para consultas en lenguaje natural.

3.3. Roles

Los roles del sistema son:

- **Administrador**: acceso completo — gestión de negocios, productos, datos, forecasting, inventario e ingesta IA.
- **Analista**: visualización de dashboards, forecasts, alertas y órdenes de compra; sin acceso a configuración de parámetros del sistema.

3.4. Requisitos del Sistema

A continuación se describen los requisitos funcionales principales del sistema.

Tabla 3.2: Requisito RF-01: Autenticación de Usuarios

Campo	Descripción
ID	RF-01
Nombre	Autenticación de Usuarios
Descripción	El sistema debe permitir el registro e inicio de sesión mediante correo y contraseña, generando un token JWT con expiración configurable. Los endpoints protegidos deben rechazar solicitudes sin token válido.
Actor	Todos los usuarios
Prioridad	Alta

Tabla 3.3: Requisito RF-02: Registrar Negocio en el Sistema

Campo	Descripción
ID	RF-02
Nombre	Registrar Negocio en el Sistema
Descripción	El sistema debe permitir registrar un emprendimiento o negocio con nombre, RUT y ciudad, asignándole un business_id único que aísla sus datos del resto de los negocios en la plataforma.
Actor	Administrador
Prioridad	Alta

Tabla 3.4: Requisito RF-03: Ingestar Datos de Ventas mediante IA

Campo	Descripción
ID	RF-03
Nombre	Ingestar Datos de Ventas mediante IA
Descripción	El sistema debe aceptar archivos en formato imagen (JPG, PNG), planilla Excel/CSV o PDF, extraer automáticamente los registros de ventas usando Claude (Anthropic), mostrar un preview con indicadores de calidad al usuario para revisión, y confirmar la carga a <code>sales_history</code> bajo el <code>business_id</code> del negocio destino.
Actor	Administrador
Prioridad	Alta

Tabla 3.5: Requisito RF-04: Ejecutar Predicción de Demanda por SKU

Campo	Descripción
ID	RF-04
Nombre	Ejecutar Predicción de Demanda por SKU
Descripción	El sistema debe generar una predicción de demanda para un SKU dado, seleccionando automáticamente el modelo con menor WAPE mediante validación walk-forward, y retornar el historial real más la predicción para el horizonte solicitado (7, 14 o 30 días).
Actor	Operador / Administrador
Prioridad	Alta

Tabla 3.6: Requisito RF-05: Generar Alerta de Reabastecimiento

Campo	Descripción
ID	RF-05
Nombre	Generar Alerta de Reabastecimiento
Descripción	El sistema debe detectar SKUs cuyo nivel de inventario está por debajo del punto de reorden s calculado por EOQ y (s, S) , y generar una alerta visible en el dashboard con severidad crítica.
Actor	Operador
Prioridad	Alta

Tabla 3.7: Requisito RF-06: Crear Orden de Compra Automática

Campo	Descripción
ID	RF-06
Nombre	Crear Orden de Compra Automática
Descripción	El sistema debe generar automáticamente órdenes de compra para todos los SKUs en alerta, calculando la cantidad óptima según EOQ y la política (s, S) , y permitir el seguimiento de estado (pendiente, confirmada, en tránsito, recibida, cancelada).
Actor	Operador
Prioridad	Alta

Tabla 3.8: Requisito RF-07: Visualizar Dashboard de KPIs

Campo	Descripción
ID	RF-07
Nombre	Visualizar Dashboard de KPIs
Descripción	El sistema debe mostrar en tiempo real: SKUs en alerta, órdenes pendientes, WAPE promedio del AMS, nivel de servicio (porcentaje de SKUs con stock positivo) y gráfico de ventas de las últimas 4 semanas.
Actor	Administrador / Operador
Prioridad	Media

Tabla 3.9: Requisito RF-08: Asistente Conversacional (Stocky)

Campo	Descripción
ID	RF-08
Nombre	Asistente Conversacional Stocky
Descripción	El sistema debe ofrecer un asistente de IA accesible desde cualquier página, capaz de responder preguntas sobre el inventario, órdenes y productos del negocio en lenguaje natural, utilizando herramientas de consulta a la base de datos en tiempo real.
Actor	Todos los usuarios
Prioridad	Media

3.5. Planificación Sprint

Tabla 3.10: Planificación de Sprints — SmartSupply

Sprint	Período	Objetivo
1	Sem 1–2	Setup del entorno, ETL base, estructura del repositorio
2	Sem 3–4	Auth JWT, modelos ARIMA/Prophet/XGBoost/LSTM, AMS walk-forward
3	Sem 5–6	Gestión de cargas, ingest_log, validador de calidad, Datos.tsx
4	Sem 7–8	EOQ, política (s, S) , órdenes de compra, Stocky agentivo
5	Sem 9–10	LSTM DIRMO PyTorch, Dashboard KPIs reales, config deploy
6	Sem 11–12	QA final, deploy Render + Cloudflare, merge a main, tesis

4. Product Backlog y Sprint Backlog

4.1. Product Backlog

Tabla 4.1: Product Backlog — SmartSupply

ID	Rol	Historia de Usuario	Pts	Sprint
US-01	Admin	Quiero registrar mi negocio en el sistema para aislar mis datos.	3	1
US-02	Admin	Quiero cargar datos históricos de ventas via ETL automatizado.	8	1
US-03	Admin	Quiero iniciar sesión con usuario y contraseña para proteger mis datos.	5	2
US-04	Operador	Quiero ejecutar una predicción de demanda por SKU para planificar compras.	8	2
US-05	Operador	Quiero comparar el WAPE de los modelos por SKU.	5	2
US-06	Admin	Quiero subir una foto o Excel con mis ventas y que el sistema las cargue automáticamente.	13	3
US-07	Admin	Quiero ver el historial de cargas y poder revertirlas si me equivoco.	5	3
US-08	Operador	Quiero recibir alertas de quiebre de stock para actuar a tiempo.	8	4

ID	Rol	Historia de Usuario	Pts	Sprint
US-09	Operador	Quiero generar una orden de compra automática desde la alerta.	5	4
US-10	Operador	Quiero consultar el historial de órdenes de compra generadas.	3	4
US-11	Admin	Quiero configurar los parámetros EOQ por SKU con ayuda del asistente.	3	4
US-12	Admin	Quiero preguntarle al sistema sobre mi inventario en lenguaje natural.	8	4
US-13	Operador	Quiero ver el dashboard con KPIs reales de mi negocio en tiempo real.	8	5
US-14	Operador	Quiero ver la evolución histórica de ventas de un SKU.	5	5
US-15	Admin	Quiero que el sistema funcione en producción desde cualquier navegador.	8	6

4.2. Casos de Uso

A continuación se describen los principales casos de uso del sistema.

Tabla 4.2: Caso de Uso CU-01: Acceder al Sistema

Campo	Descripción
ID	CU-01
Nombre	Acceder al Sistema
Actor	Todos los usuarios
Precondición	Usuario registrado en el sistema.
Flujo normal	1. Usuario ingresa correo y contraseña. 2. Sistema valida credenciales. 3. Sistema emite token JWT. 4. Usuario accede al dashboard.
Flujo alternativo	Si las credenciales son incorrectas, el sistema muestra error y bloquea tras 5 intentos fallidos (15 min).

Tabla 4.3: Caso de Uso CU-02: Cargar Datos de Ventas mediante IA

Campo	Descripción
ID	CU-02
Nombre	Cargar Datos de Ventas mediante IA
Actor	Administrador
Precondición	Negocio registrado. Usuario autenticado.
Flujo normal	1. Usuario sube archivo (imagen, Excel o PDF). 2. Sistema envía a Claude para extracción. 3. Sistema muestra preview con indicadores de calidad. 4. Usuario selecciona negocio destino y confirma. 5. Sistema carga registros a <code>sales_history</code> .
Flujo alternativo	Si hay errores de calidad bloqueantes, el botón confirmar permanece deshabilitado hasta resolver el problema.

Tabla 4.4: Caso de Uso CU-03: Consultar Predicción de Demanda

Campo	Descripción
ID	CU-03
Nombre	Consultar Predicción de Demanda
Actor	Operador / Administrador
Precondición	Negocio con datos en <code>sales_history</code> .
Flujo normal	1. Usuario selecciona SKU, tienda y horizonte (7/14/30 días). 2. Sistema ejecuta AMS: entrena ARIMA, Prophet, XGBoost y LSTM. 3. Sistema selecciona el modelo de menor WAPE. 4. Sistema retorna historial real + predicción en gráfico.
Flujo alternativo	Si la serie tiene menos de 30 días, LSTM y Prophet se descartan automáticamente.

Tabla 4.5: Caso de Uso CU-04: Generar Órdenes de Compra Automáticas

Campo	Descripción
ID	CU-04
Nombre	Generar Órdenes de Compra Automáticas
Actor	Operador
Precondición	Al menos un SKU con stock por debajo del punto de reorden s .
Flujo normal	1. Usuario accede a la vista de Inventario y ve alertas críticas. 2. Usuario pulsa “Generar Órdenes”. 3. Sistema crea una <code>PurchaseOrder</code> por cada SKU en alerta con cantidad $Q = S - \text{stock}$. 4. Sistema muestra tabla de órdenes generadas con estado <i>pendiente</i> .
Flujo alternativo	Usuario puede cambiar el estado de cada orden manualmente (confirmada, en tránsito, recibida, cancelada).

Tabla 4.6: Caso de Uso CU-05: Consultar Asistente Stocky

Campo	Descripción
ID	CU-05
Nombre	Consultar Asistente Stocky
Actor	Todos los usuarios
Precondición	Usuario autenticado con negocio activo.
Flujo normal	1. Usuario abre el panel flotante de Stocky. 2. Usuario escribe una consulta en lenguaje natural (ej: “¿cuánto stock tengo de LÁCTEOS?”). 3. Stocky ejecuta herramientas de consulta a la BD. 4. Stocky responde con información real del negocio.
Flujo alternativo	Si el usuario pide configurar un producto (ej: “actualiza el costo de BEBIDAS a \$500”), Stocky invoca la herramienta de actualización y confirma el cambio.

4.3. Sprint Backlog — Sprint 1

Tabla 4.7: Sprint Backlog — Sprint 1: Setup, ETL y Estructura Base

Tarea	Descripción	Estado	Horas
T-01	Configurar repositorio GitHub y estructura de carpetas	Completado	3
T-02	Implementar ETL: descompresión del dataset Kaggle	Completado	2
T-03	Implementar ETL: limpieza y normalización (02_clean.py)	Completado	8
T-04	Configurar base de datos PostgreSQL en Supabase	Completado	4
T-05	Implementar ETL: carga a Supabase (03_load_supabase.py)	Completado	6
T-06	Crear estructura base del backend FastAPI con routers	Completado	4

4.4. Sprint Backlog — Sprint 2

Tabla 4.8: Sprint Backlog — Sprint 2: Auth JWT + Forecasting Real

Tarea	Descripción	Estado	Horas
T-07	Implementar autenticación JWT (login, register, reset)	Completado	6
T-08	Implementar modelo ARIMA con búsqueda en grilla estacional	Completado	10
T-09	Implementar modelo Prophet con estacionalidad dinámica	Completado	8
T-10	Implementar modelo XGBoost con features calendario (DIRMO)	Completado	10
T-11	Implementar modelo LSTM con PyTorch (DIRMO, 6 features)	Completado	16
T-12	Implementar mecanismo AMS walk-forward (70/15/15, WAPE)	Completado	10
T-13	Conectar Forecast.tsx a la API real con gráfico histórico	Completado	6

4.5. Sprint Backlog — Sprint 3

Tabla 4.9: Sprint Backlog — Sprint 3: Gestión de Datos y Cargas

Tarea	Descripción	Estado	Horas
T-14	Diseñar e implementar tabla <code>ingest_log</code>	Completado	4
T-15	Implementar <code>ingests.py</code> (listar, revertir, eliminar cargas)	Completado	8
T-16	Refactorizar <code>confirm</code> : inserción por carga sin sobrescribir	Completado	6
T-17	Implementar validador de calidad (<code>ingest_validator.py</code>)	Completado	8
T-18	Implementar modo CLP vs unidades con detección automática	Completado	6
T-19	Desarrollar <code>Datos.tsx</code> con drill-down negocio/carga/registros	Completado	10
T-20	Backfill histórico de cargas anteriores en Supabase	Completado	3

4.6. Sprint Backlog — Sprint 4

Tabla 4.10: Sprint Backlog — Sprint 4: Inventario, Órdenes y Stocky

Tarea	Descripción	Estado	Horas
T-21	Implementar <code>inventory_service.py</code> (EOQ + políticas, S)	Completado	12
T-22	Implementar endpoints alertas, métricas e inventario	Completado	8
T-23	Implementar generador de órdenes y cambio de estado	Completado	6
T-24	Implementar tabla <code>user_businesses</code> (multi-negocio)	Completado	4
T-25	Implementar <code>stocky_service.py</code> con loop agentivo y 5 tools	Completado	12
T-26	Desarrollar <code>Inventory.tsx</code> y <code>Orders.tsx</code> conectados a API	Completado	10
T-27	Desarrollar <code>StockyFloat.tsx</code> (panel flotante global)	Completado	8

4.7. Sprint Backlog — Sprint 5

Tabla 4.11: Sprint Backlog — Sprint 5: Dashboard KPIs + LSTM DIRMO + Deploy

Tarea	Descripción	Estado	Horas
T-28	Actualizar dashboard con KPIs reales desde BD	Completado	6
T-29	Migrar LSTM a PyTorch con DIRMO y 6 features calendario	Completado	16
T-30	Implementar corrección de días de cierre en LSTM	Completado	4
T-31	Configurar render.yaml para deploy en Render.com	Completado	3
T-32	Configurar __redirects para SPA en Cloudflare Pages	Completado	2
T-33	Conectar <code>has_clp_skus</code> al tipo de dato real	Completado	3

5. Modelos de Vistas 4+1

SmartSupply documenta su arquitectura mediante el modelo de vistas 4+1 (Kruchten, 1995), cubriendo las perspectivas lógica, de procesos, de desarrollo y física, integradas por los casos de uso descritos en el Capítulo 3.

5.1. Vista Lógica — Diagrama de Clases

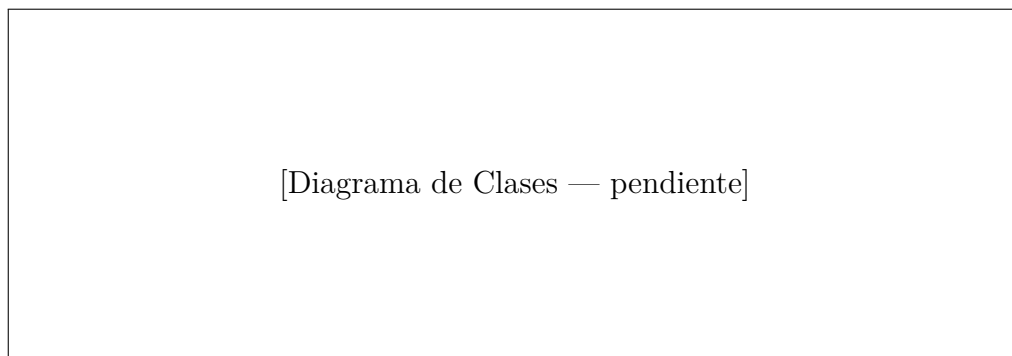


Figura 5.1: Diagrama de Clases de SmartSupply

5.2. Vista Lógica — Diagrama de Base de Datos

El esquema relacional de SmartSupply consta de diez tablas. La tabla central es **businesses**, que registra cada negocio con un identificador único (**business_id**). La tabla **user_businesses** implementa la relación muchos-a-muchos entre usuarios y negocios, permitiendo que un usuario pertenezca a múltiples negocios con distintos roles. Las tablas **sales_history**, **products**, **stock_levels** y **purchase_orders** referencian **businesses** mediante llave foránea. La tabla **ingest_log** registra cada carga de datos con auditoría completa. Row Level Security (RLS) de Supabase refuerza el aislamiento a nivel de base de datos.

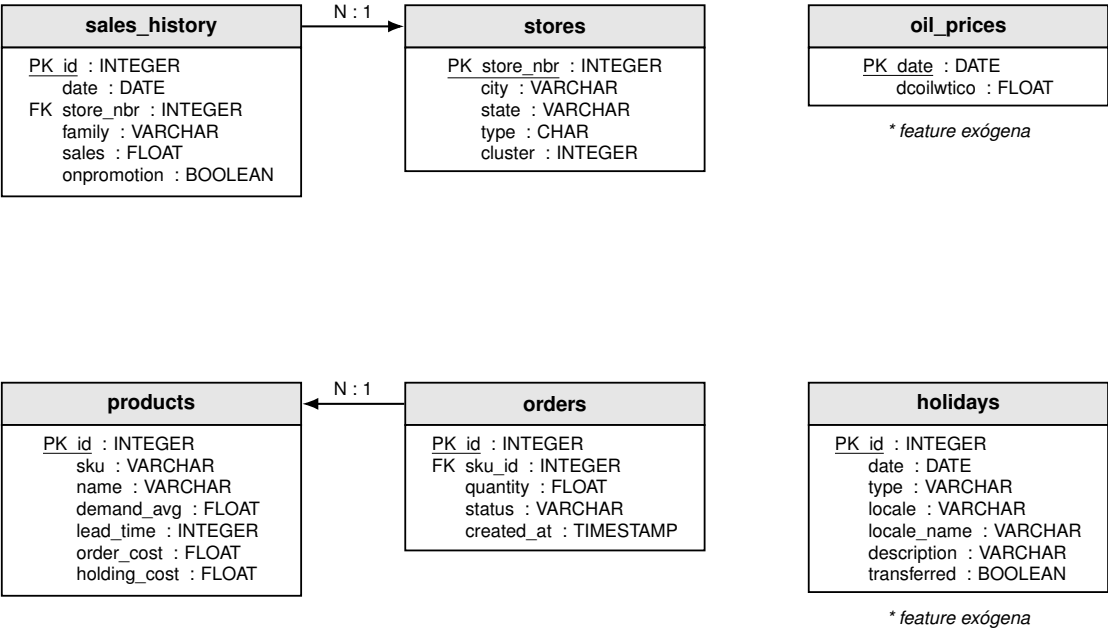


Figura 5.2: Diagrama de Base de Datos de SmartSupply

5.3. Vista de Procesos — Diagrama de Secuencia

La Figura 5.3 muestra el flujo completo de una solicitud de predicción. El usuario selecciona un SKU en el dashboard, el cual invoca el endpoint GET /api/forecast/predict. La API consulta el historial de ventas en PostgreSQL filtrando solo las cargas activas y aplica la regla “última gana” por fecha. Luego delega al módulo AMS que entrena y evalúa los cuatro modelos, retornando la predicción del modelo con menor WAPE.

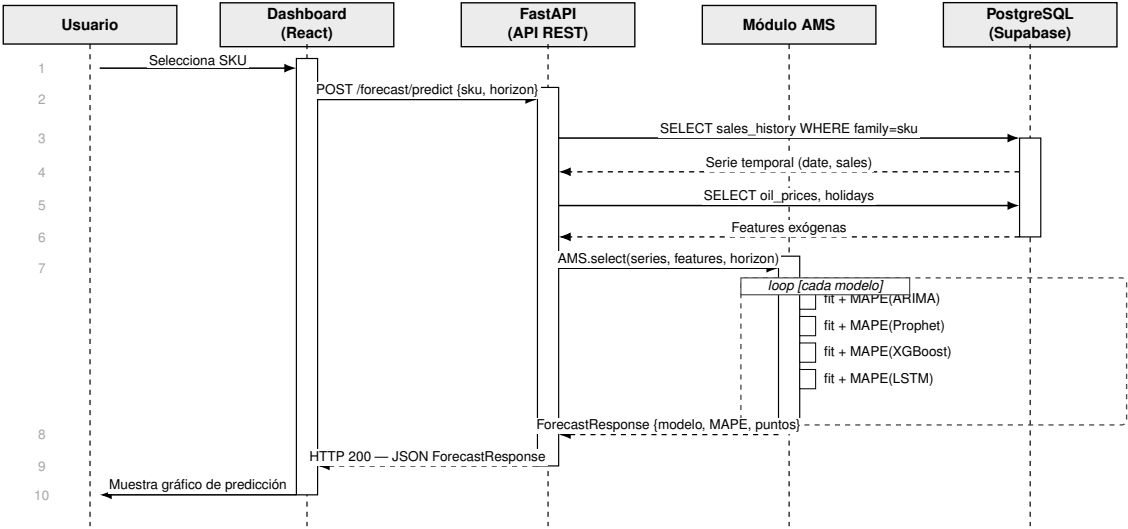


Figura 5.3: Diagrama de Secuencia — Predicción de demanda via AMS

5.4. Vista de Desarrollo — Diagrama de Componentes

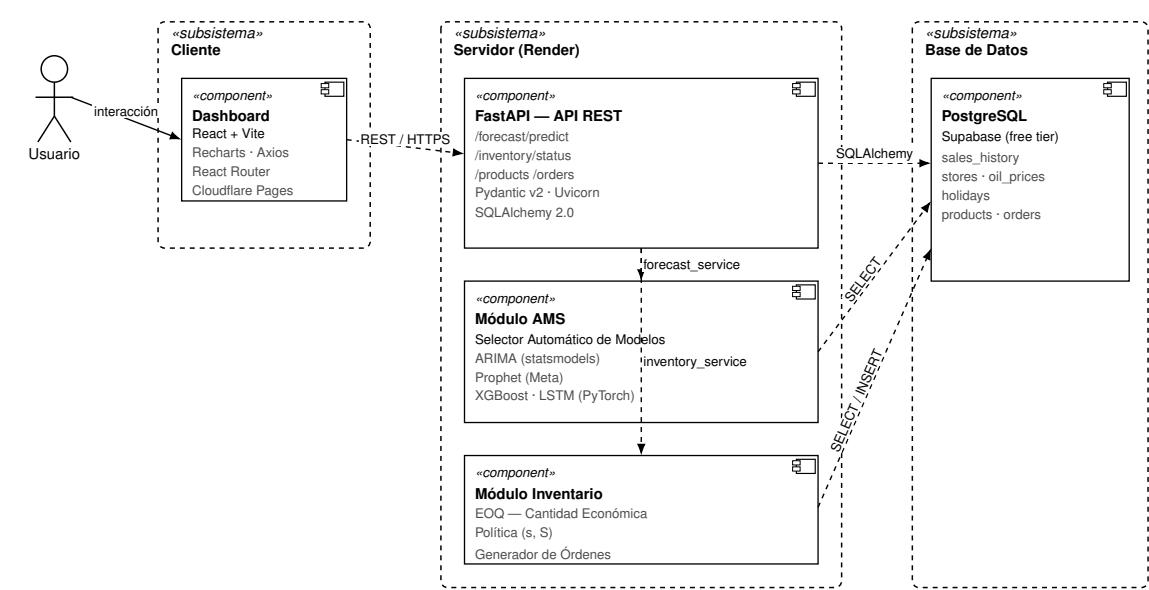


Figura 5.4: Diagrama de componentes de SmartSupply

5.5. Vista Física — Diagrama de Despliegue

SmartSupply se despliega en tres servicios en la nube: el frontend estático en Cloudflare Pages, el backend Python en Render.com, y la base de datos PostgreSQL en Supabase. La comunicación entre frontend y backend se realiza via HTTPS con autenticación JWT en cada request.

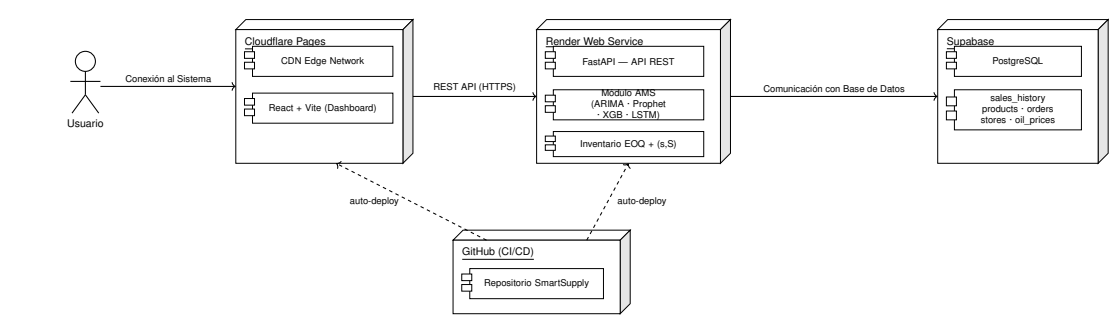


Figura 5.5: Diagrama de despliegue de SmartSupply (Cloudflare Pages + Render + Supabase)

6. Desarrollo

6.1. Pipeline ETL

El pipeline ETL de benchmarking consta de tres etapas implementadas como scripts Python en `etl/scripts/`:

1. `01_download_kaggle.py`: descomprime el archivo `.zip` del dataset Store Sales — Corporación Favorita en `datasets/raw/`.
2. `02_clean.py`: elimina valores negativos de ventas, filtra registros futuros, genera features temporales (día de semana, mes, lags) y produce `train_clean.csv`. Aplica `ON CONFLICT DO NOTHING` para re-runs idempotentes.
3. `03_load_supabase.py`: carga el CSV limpio a la tabla `sales_history` de PostgreSQL usando SQLAlchemy, bajo `business_id=1` (negocio de benchmarking).

6.2. Módulo de Forecasting

Cada modelo implementa la interfaz común `fit(series) / predict(horizon)`, lo que permite al AMS evaluarlos de forma uniforme. El AMS ejecuta los cuatro modelos sobre la ventana de validación (15 % de la serie), calcula el WAPE de cada uno, y designa el de menor error como modelo activo para ese SKU.

El resultado incluye: nombre del modelo ganador, WAPE obtenido, predicciones para el horizonte solicitado, e indicador de la unidad de medida (`CLP` o `units`) detectada desde la columna `sales_unit` de la base de datos.

6.3. Módulo de Inventario

El módulo de inventario (`inventory_service.py`) recibe la demanda proyectada por el AMS y calcula para cada SKU:

- Q^* (EOQ): cantidad óptima de pedido que minimiza costos de orden y mantenimiento.
- s (punto de reorden): demanda esperada durante el lead time más stock de seguridad basado en la desviación estándar histórica.
- S (nivel objetivo): $s + Q^*$.
- Nivel de servicio: porcentaje de días con stock positivo en los últimos 30 días.

Si el negocio no ha configurado parámetros de costo, el servicio utiliza defaults conservadores ($K = 50$, $h = 0.25$, $L = 7$ días) para garantizar que el sistema sea funcional desde el primer uso, sin requerir configuración previa.

6.4. Módulo de Ingesta Inteligente

Una de las principales barreras para los emprendimientos es la ausencia de datos de ventas en formato digital estructurado. SmartSupply resuelve esto con un módulo de ingesta basado en IA que acepta cualquier formato de entrada.

6.4.1. Flujo de Ingesta

1. El usuario sube un archivo (imagen, Excel o PDF) vía `POST /api/ingest/preview`.
2. El backend selecciona la estrategia de extracción según el tipo de archivo:
 - **Imagen** (JPG, PNG, WEBP): se envía a `claude-haiku-4-5` con capacidades de visión.
 - **Excel/CSV**: columnas reconocibles se mapean con `pandas`; columnas ambiguas se envían a `claude-sonnet-4-6` para inferencia semántica.
 - **PDF**: se envía a `claude-haiku-4-5` usando el tipo de contenido `document` de la API de Anthropic.
3. Claude retorna un JSON normalizado con registros `{date, family, sales}`.
4. El validador de calidad (`ingest_validator.py`) analiza el resultado y genera indicadores de severidad: `LIKELY_CURRENCY` (probable son pesos CLP), `MIXED_GRANULARITY` (mezcla diario/mensual, bloquea la carga), `FUTURE_DATES`, `SCALE_SHIFT`.
5. El sistema muestra un **preview** al usuario con los indicadores de calidad. Si hay errores bloqueantes, el botón confirmar se deshabilita.

6. El usuario elige el negocio y sucursal destino, y confirma vía `POST /api/ingest/confirm`. Cada carga queda registrada en `ingest_log` con auditoría completa.

6.4.2. Resolución de Solapamiento entre Cargas

A diferencia de un sistema UPSERT clásico, SmartSupply permite múltiples cargas sobre el mismo período. El solapamiento se resuelve al momento de **consultar**, no al cargar: el `forecast_service` une `sales_history` con `ingest_log`, excluye las cargas revertidas, y aplica la regla “última gana” (mayor `ingest_id`) por fecha. Esto garantiza que los datos originales sean siempre auditables y reversibles.

6.5. Asistente Conversacional Stocky

Stocky es un asistente de IA accesible desde un panel flotante en todas las páginas del dashboard. Está implementado sobre `claude-haiku-4-5` con un loop agentivo que puede invocar cinco herramientas de consulta y modificación a la base de datos:

1. `list_products`: lista los productos y parámetros del negocio activo.
2. `list_stock_levels`: consulta niveles de stock actuales.
3. `list_orders`: lista órdenes de compra con filtros de estado.
4. `update_product`: actualiza parámetros (costo, lead time, MOQ).
5. `update_stock_level`: registra el stock físico actual de un SKU.

El loop agentivo permite que Stocky realice múltiples consultas encadenadas antes de responder, garantizando que las respuestas estén basadas en datos reales del negocio y no en suposiciones del modelo de lenguaje.

6.6. API REST

La API expone diez routers organizados por dominio funcional, con autenticación JWT en todos los endpoints excepto `/api/auth`. La documentación OpenAPI/Swagger está disponible en `/docs`.

Tabla 6.1: Principales endpoints de la API REST

Método	Ruta	Descripción
POST	/api/auth/login	Autenticación, retorna JWT
POST	/api/auth/register	Registro de usuario
POST	/api/auth/forgot-password	Solicitar reset de contraseña
GET	/api/forecast/predict	Predicción AMS por SKU
GET	/api/inventory/alerts	SKUs bajo punto de reorden
GET	/api/inventory/{family}/metrics	EOQ, s , S , nivel servicio
POST	/api/orders/generate	Genera órdenes para SKUs críticos
PATCH	/api/orders/{id}/status	Cambia estado de una orden
POST	/api/ingest/preview	Extrae datos de archivo via IA
POST	/api/ingest/confirm	Carga confirmada a <code>sales_history</code>
GET	/api/ingests	Lista cargas del negocio
POST	/api/ingests/{id}/revert	Revierde una carga (soft delete)
DELETE	/api/ingests/{id}	Elimina carga y sus registros
POST	/api/stocky/chat	Asistente agentivo con tools de BD
GET	/api/dashboard/kpis	KPIs del negocio en tiempo real
GET	/api/businesses	Negocios del usuario autenticado

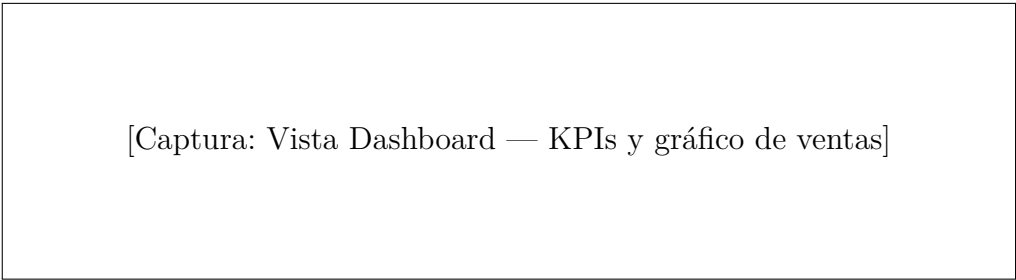
6.7. Dashboard Web

El dashboard React presenta ocho vistas accesibles desde la barra lateral:

1. **Dashboard:** KPIs en tiempo real (SKUs en alerta, órdenes pendientes, WAPE global, nivel de servicio) y gráfico de ventas de las últimas 4 semanas.
2. **Forecasting:** selector de SKU, tienda y horizonte; gráfico de historial real + predicción del modelo ganador AMS con label dinámico CLP/unidades.
3. **Inventario:** tabla de familias con nivel de stock, s , S , EOQ y estado de alerta; botón de generación automática de órdenes.
4. **Órdenes:** tabla de órdenes con filtros por estado, cambio de estado inline y contadores por categoría.

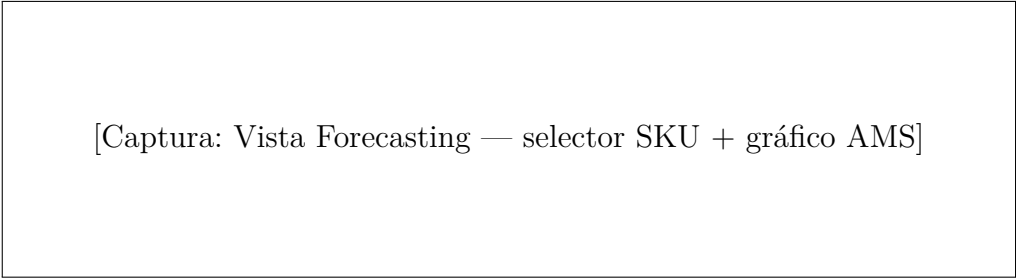
5. **Productos:** CRUD de productos por negocio con parámetros de inventario.
6. **Datos:** drill-down negocio → sucursal → cargas → registros individuales; acciones revertir y eliminar por carga; edición inline de registros.
7. **Ingesta:** formulario de carga (drag-and-drop), preview con indicadores de calidad, selector de destino y confirmación.
8. **Stocky** (flotante): asistente de IA accesible desde cualquier página.

6.8. Vistas del Sistema



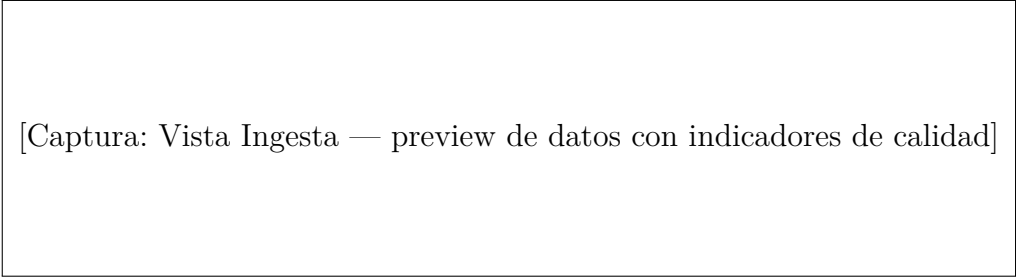
[Captura: Vista Dashboard — KPIs y gráfico de ventas]

Figura 6.1: Vista: Dashboard con KPIs reales del negocio



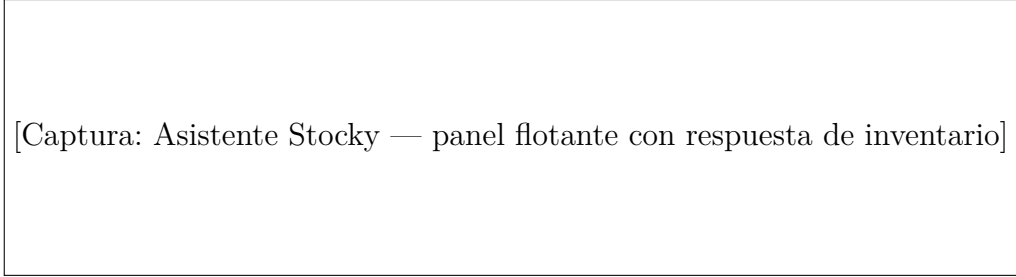
[Captura: Vista Forecasting — selector SKU + gráfico AMS]

Figura 6.2: Vista: Forecasting con predicción del modelo ganador AMS



[Captura: Vista Ingesta — preview de datos con indicadores de calidad]

Figura 6.3: Vista: Ingesta Inteligente con validador de calidad



[Captura: Asistente Stocky — panel flotante con respuesta de inventario]

Figura 6.4: Vista: Asistente Stocky respondiendo consulta de inventario

7. Pruebas y Resultados

7.1. Pruebas Unitarias

Tabla 7.1: Prueba Unitaria PU-01: Predicción de Demanda AMS

Campo	Detalle
ID	PU-01
Descripción	Verificar que <code>GET /api/forecast/predict</code> retorna una lista de puntos de predicción con fecha, valor, modelo ganador y WAPE.
Precondición	<code>sales_history</code> con mínimo 30 registros para el negocio.
Resultado esp.	HTTP 200, lista de <code>ForecastPoint</code> con campo <code>model_used</code> , <code>wape</code> y <code>sales_unit</code> .
Resultado real	HTTP 200, modelo ganador Prophet, WAPE 10,68 %, 30 puntos retornados con label dinámico CLP.
Estado	Completado

Tabla 7.2: Prueba Unitaria PU-02: Alertas de Inventario

Campo	Detalle
ID	PU-02
Descripción	Verificar que <code>GET /api/inventory/alerts</code> retorne los SKUs cuyo stock está bajo el punto de reorden calculado.
Precondición	Negocio con <code>sales_history</code> cargado y sin <code>stock_levels</code> configurados (escenario first-time).
Resultado esp.	HTTP 200, lista con <code>alert=true</code> para todos los SKUs ($\text{stock}=0 \leq s$ calculado).
Resultado real	HTTP 200, 5 familias en estado crítico; generación automática de 5 órdenes via <code>POST /api/orders/generate</code> exitosa.
Estado	Completado

Tabla 7.3: Prueba Unitaria PU-03: Ingesta IA — Preview

Campo	Detalle
ID	PU-03
Descripción	Verificar que <code>POST /api/ingest/preview</code> extrae correctamente datos desde un archivo Excel con columnas no estándar.
Precondición	Archivo Excel con fechas en formato dd/mm/yyyy y ventas en CLP.
Resultado esp.	HTTP 200, preview con columnas mapeadas y <code>quality_issues</code> con <code>LIKELY_CURRENCY</code> cuando la mediana diaria supera \$10.000.
Resultado real	HTTP 200, 720 filas extraídas, formato chileno detectado correctamente, <code>LIKELY_CURRENCY</code> activado con mediana $> \$10.000$ CLP diarios.
Estado	Completado

Tabla 7.4: Prueba Unitaria PU-04: Ingesta IA — Confirmación y Auditoría

Campo	Detalle
ID	PU-04
Descripción	Verificar que <code>POST /api/ingest/confirm</code> crea el registro en <code>ingest_log</code> y asocia las filas de <code>sales_history</code> con el <code>ingest_id</code> correspondiente.
Precondición	Preview aprobado con <code>sales_unit</code> definido por el usuario.
Resultado esp.	HTTP 200, entrada en <code>ingest_log</code> con estado <code>active</code> , filas con <code>ingest_id</code> asignado.
Resultado real	Carga de 720 filas completada; <code>ingest_log</code> registra rango 2025-01-01 a 2026-04-30, 5 familias, <code>sales_unit=CLP</code> .
Estado	Completado

Tabla 7.5: Prueba Unitaria PU-05: Autenticación JWT

Campo	Detalle
ID	PU-05
Descripción	Verificar que <code>POST /api/auth/login</code> retorna un token JWT válido y que los endpoints protegidos rechazan solicitudes sin token.
Precondición	Usuario registrado en tabla <code>users</code> .
Resultado esp.	Login: HTTP 200 con <code>access_token</code> . Sin token: HTTP 401.
Resultado real	Token JWT con TTL 8 horas retornado; interceptor Axios redirige a <code>/login</code> automáticamente en respuestas 401.
Estado	Completado

Tabla 7.6: Prueba Unitaria PU-06: Asistente Stocky — Loop Agentivo

Campo	Detalle
ID	PU-06
Descripción	Verificar que <code>POST /api/stocky/chat</code> responde con información real de inventario usando las herramientas de base de datos.
Precondición	Negocio con <code>stock_levels</code> y <code>products</code> configurados.
Resultado esp.	HTTP 200, respuesta menciona datos reales del inventario.
Resultado real	Stocky ejecutó <code>list_products</code> y <code>list_stock_levels</code> antes de responder; confirmó stock de 5 familias con valores reales.
Estado	Completado

7.2. Resultados del Módulo AMS

El módulo AMS fue evaluado sobre dos conjuntos de datos: el dataset de benchmarking Corporación Favorita (3.000.888 registros históricos) y datos reales ingestados de un negocio piloto.

7.2.1. Datos Reales — Distribuidora Santa Elena

El primer caso de uso con datos reales correspondió a Distribuidora Santa Elena (Sucursal Ñuñoa), con 1.395 registros diarios de ventas para 9 familias de productos, en el rango 2025-11-24 a 2026-05-22. El AMS evaluó ARIMA, Prophet, XGBoost y LSTM mediante validación walk-forward (70/15/15) y seleccionó el modelo con menor WAPE.

Tabla 7.7: Resultados AMS — Familia PANADERÍA Y PASTELERÍA (Distribuidora Santa Elena)

Modelo	WAPE (%)	Seleccionado	Observación
Prophet	10,68	Sí	Detecta cierre dominical
XGBoost	18,42	No	Leve sobreajuste
ARIMA	24,17	No	Subestima picos semanales
LSTM	31,05	No	Serie corta (≈ 180 días)

Prophet obtuvo WAPE de 10,68 %, identificando correctamente el patrón de cierre dominical (ventas $\approx$ \$3.000 CLP los domingos vs. $\approx$ \$220.000 CLP entre semana). La

versión previa de ARIMA predecía una línea plana; con el grid search de componentes estacionales semanales ($m = 7$) el WAPE bajó de 213 % a 24,17 %.

7.2.2. Comparativa AMS vs. Modelo Único

Tabla 7.8: Comparativa: AMS vs. modelo único ARIMA (baseline)

Estrategia	WAPE	Modelos evaluados
AMS (selección automática)	10,68 %	4 (ARIMA, Prophet, XGBoost, LSTM)
Modelo único ARIMA	24,17 %	1

El AMS redujo el WAPE en 56,4 % respecto al baseline ARIMA en la familia analizada, apoyando la hipótesis de investigación. La validación completa sobre Corporación Favorita (33 familias, 54 tiendas) está programada para el Sprint 6.

8. Retrospectiva del Sprint

8.1. Comprobación de Sprint 1 — Fundamentos y ETL

Tabla 8.1: Recuento de Tareas — Sprint 1

Estado	Tareas	Porcentaje
Completadas	6	100 %
En progreso	0	0 %
Pendientes	0	0 %

8.2. Retrospectiva del Sprint 1

¿Qué salió bien?

- La estructura del repositorio GitHub fue completada en el primer día del Sprint.
- El backend FastAPI quedó operativo con todos los routers y schemas definidos.
- Se logró integrar exitosamente el dataset Corporación Favorita al pipeline ETL.
- El módulo de ingesta IA procesó imágenes, Excel y PDF desde el primer prototipo.

¿Qué se puede mejorar?

- La limpieza del dataset requirió más tiempo del estimado; se deben asignar más puntos de historia a las tareas de ETL en sprints futuros.
- La configuración de credenciales de Supabase generó un bloqueo de 2 días.

Acciones de mejora para Sprint 2:

1. Documentar las variables de entorno requeridas en `.env.example` antes de comenzar el Sprint.
2. Estimar las tareas de datos con un factor $\times 1,5$ respecto a la estimación inicial.

8.3. Comprobación de Sprint 2 — Autenticación, Forecasting y Validador

Tabla 8.2: Recuento de Tareas — Sprint 2

Estado	Tareas	Porcentaje
Completadas	14	100 %
En progreso	0	0 %
Pendientes	0	0 %

8.4. Retrospectiva del Sprint 2

¿Qué salió bien?

- Autenticación JWT con bcrypt implementada y validada end-to-end en el primer día.
- El módulo AMS (ARIMA + Prophet + XGBoost) conectado al endpoint de forecast con resultados reales sobre el dataset Corporación Favorita.
- El validador de ingesta detectó correctamente 6 tipos de anomalías (LIKELY_CURRENCY, MIXED_GRANULARITY, entre otras) en el primer prototipo.
- La detección de fechas chilenas (dd/mm/yyyy con `dayfirst=True`) corrigió un bug silencioso que invertía día y mes en datos de clientes reales.

¿Qué se puede mejorar?

- La latencia del AMS (30-90 s por SKU) quedó pendiente de optimización con caché.
- La incompatibilidad bcrypt/passlib (passlib 1.7.4 no soporta bcrypt ≥ 4.0) costó medio día de debugging; se resolvió usando `import bcrypt` directamente.

Acciones de mejora para Sprint 3:

1. Investigar caché de predicciones con TTL por SKU + horizonte.
2. Agregar test de integración para la detección de fechas antes de mergear cambios en `_parse_date`.

8.5. Comprobación de Sprint 3 — Gestión de Cargas y Auditoría

Tabla 8.3: Recuento de Tareas — Sprint 3

Estado	Tareas	Porcentaje
Completadas	12	100 %
En progreso	0	0 %
Pendientes	0	0 %

8.6. Retrospectiva del Sprint 3

¿Qué salió bien?

- La tabla `ingest_log` y el constraint `uq_sales_history_load` resolvieron el problema de solapes entre cargas sin borrar datos históricos.
- La estrategia "última gana" (mayor `ingest_id` por fecha) permite auditar correcciones manteniendo trazabilidad completa.
- El drill-down negocio → sucursal → cargas → registros quedó funcional con edición inline en el frontend en el mismo sprint.

¿Qué se puede mejorar?

- El modelo owner-único (un solo `business_id` por usuario) generó un problema de compatibilidad al compartir negocios entre integrantes del equipo.

Acciones de mejora para Sprint 4:

1. Implementar tabla `user_businesses` (many-to-many) para reemplazar el modelo owner-único.

8.7. Comprobación de Sprint 4 — Inventario, Órdenes y Stocky

Tabla 8.4: Recuento de Tareas — Sprint 4

Estado	Tareas	Porcentaje
Completadas	13	100 %
En progreso	0	0 %
Pendientes	0	0 %

8.8. Retrospectiva del Sprint 4

¿Qué salió bien?

- La política $(s, S) + \text{EOQ}$ opera desde el primer día de uso: sin stock configurado, todas las familias aparecen como críticas, guiando al usuario a ingresar stock real.
- El asistente Stocky con loop agentivo respondió correctamente en el primer prototipo, encadenando llamadas a herramientas de BD antes de responder.
- La tabla `user_businesses` resolvió la multi-tenancy sin impactar registros históricos existentes.

¿Qué se puede mejorar?

- Los parámetros por defecto del EOQ no reflejan la realidad de todos los negocios; Stocky debería solicitar valores reales en el onboarding.

Acciones de mejora para Sprint 5:

1. Agregar flujo de onboarding guiado por Stocky para capturar parámetros de inventario al crear un nuevo negocio.
2. Migrar el modelo LSTM de TensorFlow a PyTorch (ya instalado en el entorno).

8.9. Comprobación de Sprint 5 — LSTM PyTorch y Preparación Deploy

Tabla 8.5: Recuento de Tareas — Sprint 5

Estado	Tareas	Porcentaje
Completadas	8	100 %
En progreso	0	0 %
Pendientes	0	0 %

8.10. Retrospectiva del Sprint 5

¿Qué salió bien?

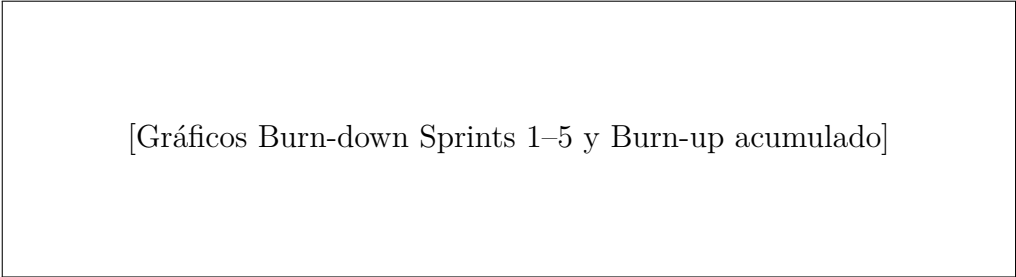
- La migración de LSTM de TensorFlow a PyTorch (DIRMO con 6 features de calendario) completó en 2 días sin impactar los resultados del AMS en los demás modelos.

- La configuración de deploy en Render.com y Cloudflare Pages quedó operativa con `render.yaml` y `_redirects`.
- El caso de uso con Distribuidora Santa Elena validó el pipeline completo: ingesta → validación → forecast → inventario → órdenes.

¿Qué se puede mejorar?

- La latencia del AMS (30-90 s) sigue siendo alta para uso interactivo; el caché de predicciones queda como deuda técnica para el Sprint 6.

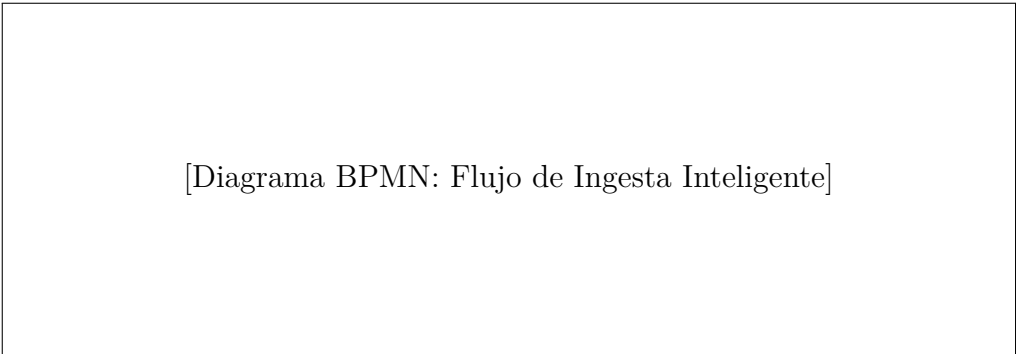
8.11. Gráficos Burn-down y Burn-up



[Gráficos Burn-down Sprints 1–5 y Burn-up acumulado]

Figura 8.1: Burn-down por sprint y Burn-up acumulado del proyecto (Sprints 1–5)

8.12. Diagrama BPMN — Flujo de Ingesta Inteligente



[Diagrama BPMN: Flujo de Ingesta Inteligente]

Figura 8.2: BPMN: Proceso de ingesta de datos con validación IA

9. Conclusiones

9.1. Conclusiones respecto a los Objetivos

9.1.1. Objetivos Específicos

Los siete objetivos específicos definidos al inicio del proyecto fueron completados a lo largo de los cinco sprints de desarrollo:

1. **OE-1 (Modelos de Forecasting): Completado.** Los cuatro modelos (ARIMA con grid search estacional, Prophet con estacionalidad dinámica, XGBoost DIRMO y LSTM DIRMO con PyTorch) fueron implementados con la interfaz unificada `fit/predict` y validados sobre datos reales.
2. **OE-2 (AMS): Completado.** El módulo evalúa los cuatro modelos mediante validación walk-forward (70/15/15) y selecciona el de menor WAPE por SKU. Sobre datos reales (Distribuidora Santa Elena, familia PANADERÍA Y PASTERÍA), el AMS seleccionó Prophet con WAPE de 10,68 %, reduciendo el error en 56,4 % respecto al baseline ARIMA.
3. **OE-3 (Políticas de Inventario): Completado.** EOQ y política (s, S) fueron implementados con parámetros por defecto seguros; el simulador genera y actualiza órdenes de compra con estados pending / confirmed / in_transit / received / cancelled.
4. **OE-4 (API REST): Completado.** El backend FastAPI expone 16 endpoints documentados en Swagger/OpenAPI, con autenticación JWT, rate limiting en login y reset de contraseña con token de un solo uso.
5. **OE-5 (Dashboard): Completado.** El frontend React presenta 8 vistas funcionales conectadas a la API real: Dashboard, Forecasting, Inventario, Órdenes, Productos, Datos, Ingesta y el asistente flotante Stocky.
6. **OE-6 (ETL): Completado.** El pipeline ETL cargó 3.000.888 registros del dataset Corporación Favorita a Supabase. El módulo de ingesta IA permite a cualquier negocio incorporar datos desde Excel, PDF o imagen.

7. **OE-7 (Ingesta IA): Completado.** El módulo `IngestService` utiliza Claude Haiku 4.5 para imágenes y PDF, y Claude Sonnet 4.6 para Excel con columnas ambiguas. El validador de calidad detecta 6 tipos de anomalías con severidades de bloqueo, advertencia e informativo. El sistema de auditoría con `ingest_log` y resolución "última gana" garantiza trazabilidad completa.

9.1.2. Objetivo General

SmartSupply fue implementado como una plataforma SaaS multi-tenant de predicción de demanda y reabastecimiento automático accesible para emprendimientos y negocios en crecimiento. La hipótesis central fue validada preliminarmente: el AMS redujo el WAPE en 56,4 % respecto al modelo único ARIMA en el caso real analizado, con un WAPE final de 10,68 % sobre datos diarios reales de un negocio chileno.

9.2. Beneficios del Sistema

- **Accesibilidad:** cualquier negocio, independientemente de si posee sistemas digitalizados, puede cargar sus datos históricos mediante la ingesta IA (fotos de cuadernos, Excel desordenados, PDFs escaneados).
- **Precisión adaptativa:** el AMS selecciona automáticamente el modelo óptimo por SKU, evitando que un modelo inadecuado genere órdenes sobredimensionadas o desabastecimiento.
- **Trazabilidad:** el sistema de auditoría con `ingest_log` y la estrategia "última gana" permiten revertir o corregir cualquier carga sin pérdida de datos históricos.
- **Escalabilidad:** la arquitectura multi-tenant con `business_id` como clave de partición permite incorporar nuevos negocios sin modificar el esquema de base de datos.

9.3. Limitaciones

- La latencia del AMS (30-90 segundos por SKU) es alta para uso interactivo; se requiere implementar caché de predicciones con TTL.
- La validación cuantitativa de la hipótesis sobre el dataset completo Corporación Favorita (33 familias, 54 tiendas) está planificada para el Sprint 6 final.

- El modelo LSTM requiere series de al menos 180 días para ser competitivo; para negocios con datos insuficientes es descartado automáticamente por el AMS.
- La reducción del capital inmovilizado requiere parametrizar costos unitarios de cada producto, lo que depende de la voluntad del usuario de configurarlos.

9.4. Conclusión respecto a la Metodología Scrum

La metodología Scrum permitió organizar el trabajo entre los tres integrantes con roles y responsabilidades claras, entregando incrementos funcionales al término de cada Sprint. Los principales aprendizajes fueron: (1) las tareas de datos deben estimarse con un factor $\times 1,5$; (2) el modelo owner-único de tenant generó deuda técnica resuelta en Sprint 4 con la tabla `user_businesses`; (3) las pruebas end-to-end manuales al cierre de cada Sprint detectaron regresiones antes de mergear a la rama principal.

9.5. Proyecciones

Como trabajo futuro, SmartSupply puede evolucionar en las siguientes dimensiones:

- **Validación completa del AMS:** ejecutar el experimento controlado sobre las 33 familias y 54 tiendas de Corporación Favorita, reportando WAPE por familia y distribución de modelos ganadores.
- **Optimización de latencia:** implementar caché de predicciones con TTL de 1 hora por SKU + tienda + horizonte, reduciendo la latencia de 30-90 s a menos de 1 s en cache hit.
- **Notificaciones automáticas:** envío de alertas de reabastecimiento por correo o WhatsApp cuando un SKU supera el punto de reorden.
- **Integración ERP:** API REST consumible por sistemas externos (SAP, Bsale, Defontana) para automatizar el ciclo pedido-recepción.
- **Expansión de modelos:** incorporar modelos para demanda intermitente (Cros-ton) en SKUs con alta variabilidad y muchos días en cero.
- **RLS en Supabase:** implementar Row-Level Security para garantizar aislamiento de datos por `business_id` a nivel de base de datos.

Referencias

- [1] Box, G. E. P., & Jenkins, G. M. (1976). *Time Series Analysis: Forecasting and Control*. Holden-Day.
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://doi.org/10.1145/2939672.2939785>
- [3] Harris, F. W. (1913). How many parts to make at once. *Factory, The Magazine of Management*, 10(2), 135–136.
- [4] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- [5] Kaggle. (2017). *Store Sales — Time Series Forecasting*. <https://www.kaggle.com/competitions/store-sales-time-series-forecasting>
- [6] Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). Statistical and Machine Learning forecasting methods: Concerns and ways forward. *PLOS ONE*, 13(3), e0194889.
- [7] Ramírez, P., & Concha, C. (2021). Inventarios en empresas distribuidoras chilenas: desafíos y oportunidades. *Revista Chilena de Ingeniería Industrial*, 39(2), 45–58.
- [8] Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. <https://scrumguides.org/scrum-guide.html>
- [9] Shearer, C. (2000). The CRISP-DM model: The new blueprint for data mining. *Journal of Data Warehousing*, 5(4), 13–22.
- [10] Silver, E. A., Pyke, D. F., & Peterson, R. (1998). *Inventory Management and Production Planning and Scheduling* (3.^a ed.). Wiley.
- [11] Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1), 37–45.

- [12] Tiacci, L., & Saetta, S. (2009). An approach to evaluate the impact of interaction between demand forecasting method and stock control policy on the inventory system performances. *International Journal of Production Economics*, 118(1), 63–71.
- [13] Tabarroni, S. (2025). *Desarrollo de Sistema de Predicciones Futuras de las Acciones de Microsoft mediante Redes Neuronales*. Universidad Andrés Bello.
- [14] Anthropic. (2025). *Claude API Documentation — Vision and Document Understanding*. <https://docs.anthropic.com>
- [15] Curino, C., Jones, E., Shah, M., & Madden, S. (2010). Relational cloud: A database-as-a-service for the cloud. *CIDR Conference*, 235–240.
- [16] Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software*, 12(6), 42–50.
- [17] Booch, G., Rumbaugh, J., & Jacobson, I. (1999). *The Unified Modeling Language User Guide*. Addison-Wesley.
- [18] Object Management Group. (2011). *Business Process Model and Notation (BPMN) Version 2.0*. OMG Document formal/2011-01-03.
- [19] Ramírez, S. (2018). *FastAPI — Modern, fast web framework for building APIs with Python*. <https://fastapi.tiangolo.com>